

NOSQL: CIAEC49

Unit-IV

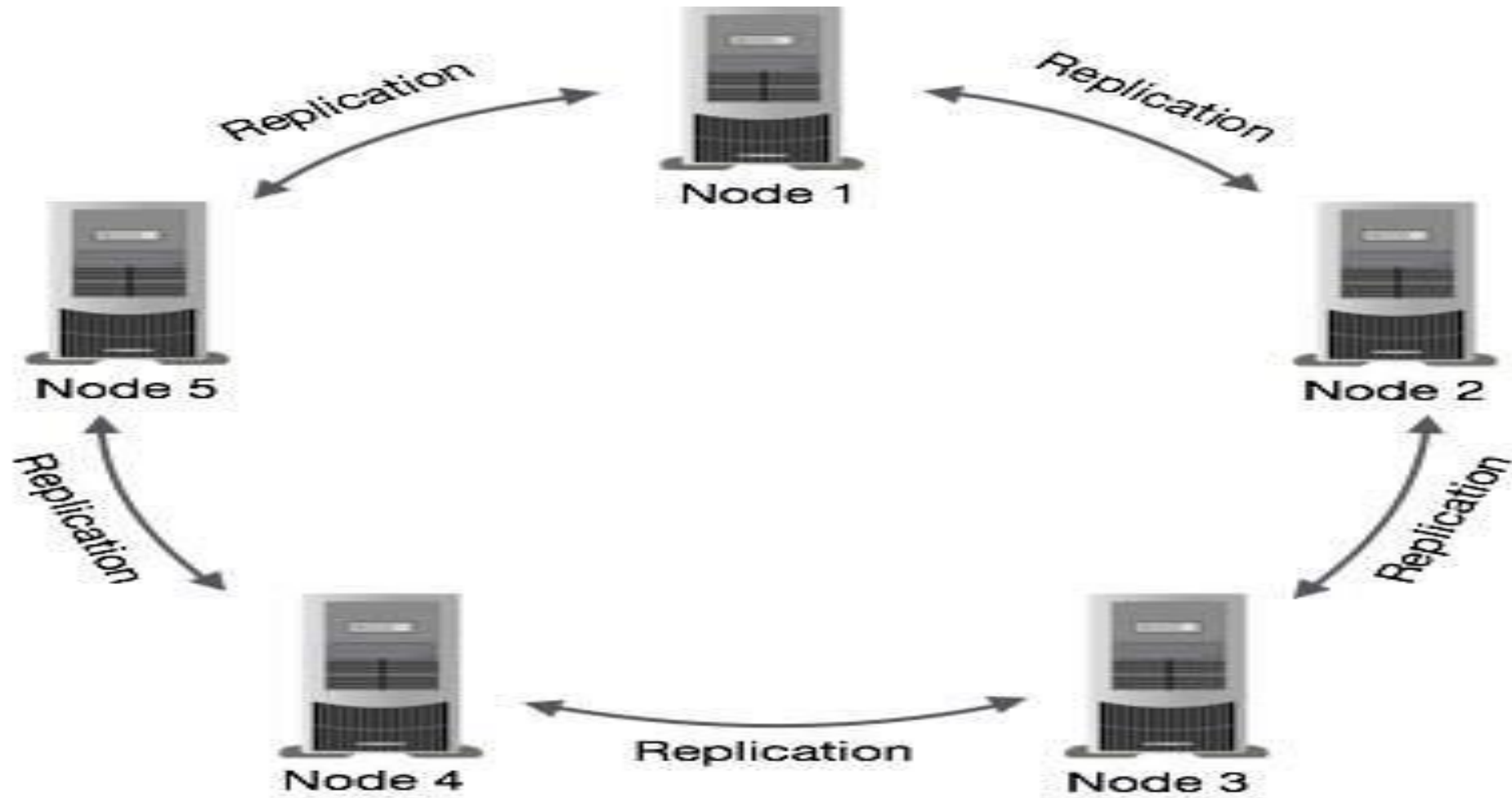
Cassandra

Cassandra - Architecture

- **Cassandra - Architecture**
- The design goal of Cassandra is to handle big data workloads across multiple nodes without any single point of failure. Cassandra has peer-to-peer distributed system across its nodes, and data is distributed among all the nodes in a cluster.
- All the nodes in a cluster play the same role. Each node is independent and at the same time interconnected to other nodes.
- Each node in a cluster can accept read and write requests, regardless of where the data is actually located in the cluster.
- When a node goes down, read/write requests can be served from other nodes in the network.

Cassandra Architecture and Replication Factor Strategy

- Data Replication in Cassandra
- In Cassandra, one or more of the nodes in a cluster act as replicas for a given piece of data. If it is detected that some of the nodes responded with an out-of-date value, Cassandra will return the most recent value to the client. After returning the most recent value, Cassandra performs a **read repair** in the background to update the stale values.
- The following figure shows a schematic view of how Cassandra uses data replication among the nodes in a cluster to ensure no single point of failure.



Features of Cassandra

Features of Cassandra

- Cassandra has become so popular because of its outstanding technical features. Given below are some of the features of Cassandra:
- **Elastic scalability** – Cassandra is highly scalable; it allows to add more hardware to accommodate more customers and more data as per requirement.
- **Always on architecture** – Cassandra has no single point of failure and it is continuously available for business-critical applications that cannot afford a failure.
- **Fast linear-scale performance** – Cassandra is linearly scalable, i.e., it increases your throughput as you increase the number of nodes in the cluster. Therefore it maintains a quick response time.
- **Flexible data storage** – Cassandra accommodates all possible data formats including: structured, semi-structured, and unstructured. It can dynamically accommodate changes to your data structures according to your need.
- **Easy data distribution** – Cassandra provides the flexibility to distribute data where you need by replicating data across multiple data centers.
- **Transaction support** – Cassandra supports properties like Atomicity, Consistency, Isolation, and Durability (ACID).
- **Fast writes** – Cassandra was designed to run on cheap commodity hardware. It performs blazingly fast writes and can store hundreds of terabytes of data, without sacrificing the read efficiency

Components of Cassandra

Components of Cassandra

- The key components of Cassandra are as follows –
- **Node** – It is the place where data is stored.
- **Data center** – It is a collection of related nodes.
- **Cluster** – A cluster is a component that contains one or more data centers.
- **Commit log** – The commit log is a crash-recovery mechanism in Cassandra. Every write operation is written to the commit log.
- **Mem-table** – A mem-table is a memory-resident data structure. After commit log, the data will be written to the mem-table. Sometimes, for a single-column family, there will be multiple mem-tables.
- **SSTable** – It is a disk file to which the data is flushed from the mem-table when its contents reach a threshold value.
- **Bloom filter** – These are nothing but quick, nondeterministic, algorithms for testing whether an element is a member of a set. It is a special kind of cache. Bloom filters are accessed after every query.

Replication strategy

- Different types of Replication strategy class options supported by Cassandra are the following:
 1. Simple Strategy
 2. Local Strategy
 3. Network Topology Strategy
- These are explained as following below.

1. Simple Strategy

- It is a simple strategy that is recommended for multiple nodes over multiple racks in a single data center.
- Let's consider taking an example, `strategy_demo` is a keyspace name in which class is `SimpleStrategy` and `replication_factor` is 2 which simply means there are two redundant copies of each row in a single data center..

Creating a keyspace

- An object called a keyspace is used to store user-defined types and column families. A keyspace is similar to an RDBMS database in that it includes indexes, user-defined types, indexes, replication factors, strategy, column families, data center awareness, etc.
- `CREATE KEYSPACE cluster1 WITH`
- `replication = {'class': 'SimpleStrategy', 'replication_factor' : 2};`

2. Local Strategy

- It is the strategy in which we will use a replication strategy for internal purposes such that is used for system and sys_auth keyspaces are internal keyspaces. In Cassandra internal keyspaces implicitly handled by Cassandra's storage architecture for managing authorization and authentication.
- It is not permissible to creating keyspace with LocalStrategy class if we will try to create such keyspace then it would give an error like "LocalStrategy is for Cassandra's internal purpose only".

3. Network Topology Strategy

- It is the strategy in which we can store multiple copies of data on different data centers as per need. This is one important reason to use Network Topology Strategy when multiple replica nodes need to be placed on different data centers.
- Let's consider an example, cluster1 is a keyspace name in which Network Topology Strategy is a replication strategy and there are two data centers one is east with RF(Replication Factor) = 2 and second is west with RF(Replication Factor) = 3.

- Cassandra Query Language
- Users can access Cassandra through its nodes using Cassandra Query Language (CQL). CQL treats the database (**Keyspace**) as a container of tables. Programmers use **cqlsh**: a prompt to work with CQL or separate application language drivers.
- Clients approach any of the nodes for their read-write operations. That node (coordinator) plays a proxy between the client and the nodes holding the data.

Write and read operations

- **Write Operations**
- Every write activity of nodes is captured by the **commit logs** written in the nodes. Later the data will be captured and stored in the **mem-table**. Whenever the mem-table is full, data will be written into the **SStable** data file. All writes are automatically partitioned and replicated throughout the cluster. Cassandra periodically consolidates the SSTables, discarding unnecessary data.
- **Read Operations**
- During read operations, Cassandra gets values from the mem-table and checks the bloom filter to find the appropriate SStable that holds the required data.

CREATE KEYSPACE

```
CREATE KEYSPACE cluster1 WITH replication = {'class':  
NetworkTopologyStrategy', 'replication_factor':1};
```

To verify all internal existing keyspaces used the following CQL query given below.

```
select * from system_schema.keyspaces where keyspace_name =  
'cluster1';
```

index ^	keyspace_name	durable_writes	graph_engine	replication
0	cluster1	true		{"caas-dc":"1","class":"org.apache.cassandra.locator.NetworkTopologyStrategy"}

Table-2:

create table tb2

```
(  
  Id int primary key,  
  event text,  
  team_name text  
);
```

Let's verify the keyspace schema of cluster1 by using the following CQL query.

How to Create Table in Cassandra

- Create table KeyspaceName.TableName
- (
• ColumnName DataType,
• ColumnName DataType,
• ColumnName DataType
• Primary key(ColumnName)
•) with PropertyName=PropertyValue

- 1. Primary key: There are two types of primary key.
- Single Primary Key: Single primary key is specified by the following syntax.
- **Syntax:**
- Primary key (ColumnName)
- In the single primary key, there is only a single column.
- That column is also called partitioning key.
- Data is partitioned on the basis of that column.
- Data is spread on different nodes on the basis of the partition key.

- 2. Compound Primary Key: Compound primary key is specified by the following syntax.
- **Syntax:**
- Primary key(ColumnName1,ColumnName2 . . .)
- In above syntax, ColumnName1 is the partitioning key and ColumnName2 is the Clustering key. Data will be partitioned on the basis of ColumnName1 and data will be clustered on the basis of ColumnName2. Clustering is the process that sorts data in the partition.

- 3. Compound Partitioning key: Compound partitioning key is specified by the following syntax.
- **Syntax:**
- Primary
Key((ColumnName1,ColumnName2),ColumnName3...))

- In syntax, ColumnName1 and ColumnName2 are the compound partition key. Data will be partitioned on the basis of both columns ColumnName1 and ColumnName2 and data will be clustered on the basis of the ColumnName3. If you have too much data on the single partition. Then, compound partitioning key is used. Compound partitioning key is used to create multiple partitions for the data.

- **With Clause**

“With clause” is used to specify any property and its value for the defined table.

For example,

if you want to compress Cassandra table data.

You can set compression property by specifying compression algorithm property value in “With clause.”

Example

Here is the execution of the command ‘Create table’ that will create table name ‘Student’ in the keyspace ‘University.’

Create Table

- To verify all the tables for a specific existing keyspace then used the following CQL query given below.

```
create table tb1  
(  
  Id int primary key,  
  name text,  
  city text  
);
```

Table-2:

```
create table tb2
```

```
(
```

```
  Id int primary key,
```

```
  event text,
```

```
  team_name text
```

```
);
```

Let's verify the keyspace schema of cluster1 by using the following CQL query.

- **System and system_auth Keyspaces:**

The system keyspace contains information about available column families, columns, and clusters. The system_auth keyspace mainly contains authentication information, user credentials, and permissions.


```
cqlsh> Create table University.Student  
..  
.. (  
.. RollNo int,  
.. Name text,  
.. dept text,  
.. primary key(RollNo)  
.. );
```

command to
create table

column name
and data types

primary key
RollNo

Table name
student

- After successful execution of the command 'Create table', table 'Student' will be created in the keyspace 'University' with columns RollNo, Name and dept.
- RollNo is the primary key. RollNo is also a partition key. All the data will be in the single partition.

Cassandra Alter table

- Command 'Alter Table' is used to drop column, add a new column, alter column name, alter column type and change the property of the table.
- Syntax
- Following is the syntax of command 'Alter Table.'

```
cqlsh> Alter Table University.Student  
... Add Semester int;
```

command to
alter table

column name and
data type to be added

table name to
be altered

Contd..

- After successful execution of the command 'Alter Table', a new column 'Semester' with 'int' data type will be added to the table Student.
- Here is the screenshot that shows the updated Student table.

```
cqlsh> desc University.Student;
CREATE TABLE university.student (
  rollno int PRIMARY KEY,
  dept text,
  name text,
  semester int
) WITH bloom_filter_fp_chance = 0.01
AND caching = '{"keys":"ALL", "rows_per_partition":"NONE"}'
AND comment = ''
AND compaction = {'class': 'org.apache.cassandra.db.compaction.SizeTieredCompactionStrategy'}
AND compression = {'sstable_compression': 'org.apache.cassandra.io.compress.LZ4Compressor'}
AND dclocal_read_repair_chance = 0.1
AND default_time_to_live = 0
AND gc_grace_seconds = 864000
AND max_index_interval = 2048
AND memtable_flush_period_in_ms = 0
AND min_index_interval = 128
AND read_repair_chance = 0.0
AND speculative_retry = '99.0PERCENTILE';

cqlsh> _
```

new added
column

Cassandra Drop Table

- **Cassandra Drop Table**
- Command 'Drop table' drops specified table including all the data from the keyspace. Before dropping the table, Cassandra takes a snapshot of the data not the schema as a backup.
- Drop Table KeyspaceName.TableName
- Here is the snapshot of the executed command 'Drop Table' that will drop table Student from the keyspace 'University'.

Drop Table

```
cqlsh> Drop table University.Student;
```

command to
drop table

table name to
be dropped

- After successful execution of the command 'Drop Table', table Student will be dropped from the keyspace University.
- Here is the snapshot that shows the error returned by the Cassandra when tried to access Student table that does not exist.

```
cqlsh> desc university.Student;  
  
'student' not found in keyspace 'university'  
cqlsh>
```

Cassandra Truncate Table

- Cassandra Truncate Table
- Command 'Truncate table' removes all the data from the specified table.
- Before truncating the data, Cassandra takes the snapshot of the data as a backup.
- Syntax
- Truncate KeyspaceName.TableName

- **Example**
- There are three records in the table Student. These are the records in the table

rollno	dept	name	semester
1	CS	Clark	2
2	CS	Michael	2
3	CS	Jason	2

- Here is the snapshot of the executed command 'Truncate table' that will remove all the data from the table Student.

```
cqlsh> truncate University.Student;
```

command to
truncate table

table name to
be deleted

Contd..

- After successful execution of the command 'Truncate Table', all the data will be removed from the table Student.
- Here is the snapshot of the database state where there are no records in the table Student.

```
rollno | dept | name | semester
-----+-----+-----+-----
(0 rows)
```

S.No.	CASSANDRA	RDBMS
1.	Cassandra is a high performance and highly scalable distributed NoSQL database management system.	RDBMS is a Database management system or software which is designed for relational databases.
2.	Cassandra is a NoSQL database.	RDBMS uses SQL for querying and maintaining the database.
3.	It deals with unstructured data.	It deals with structured data.
4.	It has a flexible schema.	It has fixed schema.
5.	Cassandra has peer-to-peer architecture with no single point of failure.	RDBMS has master-slave core architecture means a single point of failure.
6.	Cassandra handles high volume incoming data velocity.	RDBMS handles moderate incoming data velocity.
7.	In RDBMS there is limited data source means data come from many locations.	In Cassandra there are various data source means data come from one/few location.
8.	It supports simple transactions.	It supports complex and nested transactions.
9.	In Cassandra the outermost container is Keyspace.	In RDBMS the outermost container is database.
10.	Cassandra follows decentralized deployments.	RDBMS follows centralized deployments.
11.	In Cassandra data written in many locations.	In RDBMS mainly data are written in one location.
12.	In Cassandra row represents a unit of replication.	In RDBMS row represents a single record.
13.	In Cassandra column represents a unit of storage.	In RDBMS column represents an attribute.
14.	In Cassandra, relationships are represented using collections.	In RDBMS relationships are represented using keys and join etc.

Redis

1. Introduction to Redis

- Redis (Remote Dictionary Server) is an open-source, in-memory data structure store used as a database, cache, and message broker. It is known for its **high speed, simplicity, and powerful data structure support**.

◆ Why Redis?

- Traditional databases store data on disk, whereas Redis stores data **in-memory**, making it much **faster**.
- It supports **persistence**, so data is not lost after a reboot.
- Used in applications that require **real-time** response, like leaderboards, session storage, and caching.

◆ Common Use Cases:

- ✓ **Caching** (e.g., storing frequently accessed data)
- ✓ **Session Management** (e.g., user sessions in web applications)
- ✓ **Message Queueing** (e.g., real-time notifications)
- ✓ **Real-time Analytics** (e.g., tracking website visits)
- ✓ **Leaderboard Management** (e.g., ranking players in games)

Features of Redis

2. Features of Redis

Redis is a popular choice for developers due to its rich feature set:

High Performance

Can process **millions of requests per second** due to in-memory storage.

Rich Data Structures

Supports multiple data structures beyond just key-value pairs.

Persistence

Redis provides **two types of persistence**:

RDB (Redis Database) – Periodic snapshots of data.

AOF (Append-Only File) – Logs every write operation for durability.

Replication (Master-Slave Architecture)

Allows **data replication across multiple servers** for high availability.

Clustering & Partitioning

Redis can distribute data across multiple servers (horizontal scaling).

Publish/Subscribe Messaging

Allows real-time messaging between different services using the **Publish-Subscribe (Pub/Sub) model**.

Security

Provides **authentication**, access control, and TLS encryption.

Data Type of Redis

Redis supports multiple data types, making it more powerful than traditional key-value stores.

Data Type	Description	Example Use Case
Strings	Stores simple key-value pairs	Caching, Counters
Lists	Ordered collection of strings	Messaging queues, Timelines
Sets	Unordered collection of unique elements	Tags, Friends list
Hashes	Store key-value pairs like a dictionary	User profiles
Sorted Sets (Zsets)	Like sets but with ranking (score)	Leaderboards, Ranking systems
Bitmaps	Bit-level operations on strings	User activity tracking
HyperLogLogs	Approximate count of unique elements	Counting website visits

4. Redis Keys

Keys in Redis are unique identifiers used to store and retrieve values.

- **Key Naming Rules**

Can be any string but should follow a pattern like:

user:1001:name

session:abc123

product:56789:price

Avoid **too many large keys**, as Redis is in-memory.

SET name "Alice" # Create a key

GET name # Retrieve the value

DEL name # Delete the key

EXISTS name # Check if key exists (returns 1 if exists, 0 if not)

EXPIRE name 60 # Set key to expire in 60 seconds

TTL name # Check time-to-live of key

RENAME old_key new_key # Rename a key

KEYS * # List all keys (not recommended for large datasets)

4. Redis Keys

5

. Redis Strings (Most Common Data Type)

Strings in Redis can store:

- Text
- Numbers
- Binary data (e.g., images, serialized objects)

String Commands

- SET mykey "Hello Redis" # Store a string
- GET mykey # Retrieve the value
- APPEND mykey " World" # Append to a string

Number of Operations

- STRLEN mykey # Get string length
- SET counter 10 # Initialize a counter
- INCR counter # Increment (result: 11)
- INCRBY counter 5 # Increment by 5 (result: 16)
- DECR counter # Decrement (result: 15)
- DECRBY counter 3 # Decrement by 3 (result: 12)